HBTXR: Algorithm–Hardware Co-Designed Hybrid Eye Tracking for On-Device Extended Reality

Anonymous Author(s)

***Abstract*—Real-time eye tracking on extended reality (XR) devices must satisfy tight latency, energy, memory, and form-factor constraints while preserving robust pupil localization and gaze estimation. Prior work has often optimized either algorithms or accelerators in isolation, which leads to mismatches between model behavior and deployment cost. We present HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework that unifies frame-guided Search, event-guided Track, runtime scheduling, and accelerator mapping in one system pipeline. At the algorithm level, HBTXR introduces a time-synchronous and geometry-stable hybrid supervision flow, a modality-aware transformer whose Search path executes the full backbone while the Track path exits after the front half of the shared backbone, and a runtime-aware scheduler that governs mode switching from quality signals. At the hardware level, HBTXR adopts quantization-aware operator reformulation, table-driven non-linear approximation, and a streaming multi-stage pipeline that jointly optimizes intra-layer dataflow and cross-layer coupling. Experimental results show that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68°, and an end-to-end Track-mode latency of 0.57 ms, while maintaining 97.8% Search success rate and 1754 Hz effective update rate in Track mode. These results suggest that HBTXR offers a practical system point for accurate, low-latency, and deployment-oriented XR eye tracking.**

***Index Terms*—Eye tracking, extended reality, hybrid sensing, event camera, transformer accelerator, FPGA, algorithm–hardware co-design.**

I. Introduction

Eye tracking has become a core enabling function for extended reality (XR), supporting gaze-based interaction, foveated rendering, user-state understanding, and privacy-preserving on-device perception [1], [2]. In practice, however, XR eye tracking must satisfy three requirements simultaneously: it must remain semantically robust under appearance changes, temporally responsive under rapid eye motion, and efficient enough to run on embedded hardware with strict power and memory budgets. This combination makes XR eye tracking fundamentally a system-level design problem rather than a stand-alone perception problem.

Existing approaches reveal a structural trade-off. Frame-based methods are strong in semantic recovery and calibration-aware regression, but repeated dense-image processing introduces high bandwidth and latency cost [3]. Event-based methods offer sparse sensing and high temporal resolution, but their reliability degrades under weak-event regimes, drift accumulation, or reinitialization failures [4]–[6]. Hybrid methods improve this balance, yet many still treat fusion, runtime mode switching, and hardware deployment as loosely coupled stages [7].

This paper addresses that gap through HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system for on-device XR. In contrast to monolithic multimodal regression, HBTXR explicitly separates *Search* and *Track*. Search is frame-guided and refresh-oriented, designed for robust relocalization and anchor recovery. Track is event-guided and state-persistent, designed for low-latency residual update around the current pupil estimate. This decomposition is reflected not only in the model architecture, but also in the supervision flow, training pipeline, runtime scheduler, and hardware architecture. In particular, Search executes the full shared backbone for global refresh, whereas Track exits after the front half of the backbone and forwards the intermediate representation directly to the Track head.

Fig. 1 summarizes the overall design. HBTXR first constructs time-synchronous supervision so that frame and event inputs share one stable ellipse-centric target. It then employs a modality-aware transformer with a shared backbone and decoupled heads. A runtime scheduler switches between Search and Track according to confidence, similarity, event density, and eye-state validity. Finally, a CPU–FPGA deployment architecture maps the shared backbone and lightweight heads to differentiated execution paths. In this way, HBTXR follows the AHCO-style principle of internalizing hardware constraints during algorithm design while tuning the hardware around the actual runtime semantics of the algorithm [11].

The key contributions of this work are summarized as follows.

- We reformulate hybrid eye tracking as a Search–Track problem and introduce a time-synchronous, geometry-stable supervision contract for aligning frame, event, and state targets.
- We design a modality-aware transformer with shared backbone and decoupled Search, Event, Track, and Mask heads, where Search uses the full backbone and Track uses only its front half before early exit.
- We develop a runtime-aware scheduler that explicitly manages Search mode, Track mode, and mode switching according to quality assessment signals.
- We present a deployment-oriented accelerator design with quantized operators, table-driven non-linear approximation, streaming execution, multi-stage pipeline balance, and CPU–FPGA co-scheduling.

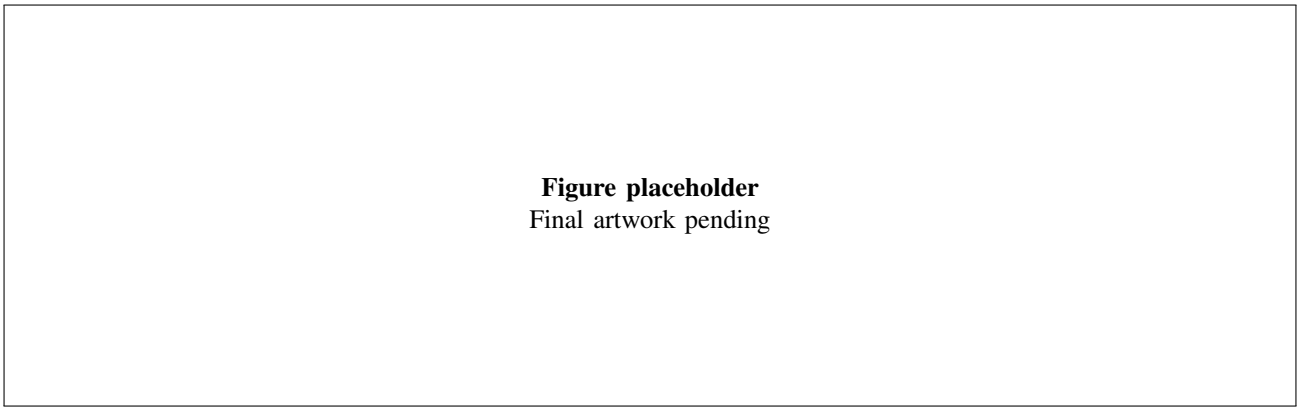

Fig. 1. Overview of HBTXR. The proposed framework combines time-synchronous hybrid supervision, a modality-aware Search/Track transformer, a runtime-aware scheduler, and a deployment-oriented FPGA architecture.

II. Related Works

A. Frame-based Eye Tracking

Frame-based eye tracking remains the dominant paradigm in near-eye and head-mounted systems because grayscale or infrared frames preserve strong semantic structure for pupil localization, iris fitting, and gaze regression [1], [3]. These methods are particularly effective for global relocalization and calibration-aware estimation. However, their strengths come with dense sensing and repeated full-frame processing, which translate into high memory traffic, bandwidth overhead, and limited responsiveness under rapid eye motion.

B. Event-based Eye Tracking

Event-based eye tracking exploits asynchronous brightness-change sensing to achieve sparse and high-temporal-resolution processing. Prior work has shown that event-driven methods are attractive for low-latency update during fast saccades and for reducing redundant processing during fixation [4]–[6]. Nonetheless, event-only pipelines remain weaker in semantic stabilization, blink recovery, and long-horizon relocalization, especially when event density becomes low or the tracker drifts away from the valid pupil contour.

C. Hybrid Frame-Event Eye Tracking

Hybrid eye tracking combines frame-based semantic recovery with event-based temporal responsiveness. Representative systems demonstrate that relocalization can be executed on low-rate frames while event streams support high-rate update [7]. This direction is promising for XR because it aligns sensing modality with tracking phase. However, many hybrid systems still leave the relationship between fusion strategy, runtime mode policy, and hardware mapping under-specified.

D. Deployment-Oriented and Hardware Accelerator

Deployment-oriented eye tracking has been studied through frame-centric accelerators, event-centric sparse designs, and ASIC-style low-power trackers [8]–[10]. These works confirm that eye tracking is not only a perception problem but also a hardware-design problem. In parallel, algorithm–hardware co-optimization has been shown to be effective in edge vision systems by integrating model design, quantization, and accelerator mapping in one workflow [11]. The remaining gap for hybrid XR eye tracking is a unified framework that co-designs Search/Track semantics, transformer inference, and deployment behavior together.

E. Summary

The literature suggests four conclusions. First, frame-centric methods remain the most reliable choice for global refresh. Second, event-centric methods remain the best candidate for ultra-low-latency local update. Third, hybrid systems are compelling only when their operating modes are made explicit. Fourth, deployment efficiency improves when hardware constraints are internalized during model design. HBTXR is built precisely around these conclusions.

III. Proposed HBTXR Algorithm

A. Framework Design

1) Problem Formulation: HBTXR operates on a hybrid sensing input composed of a frame stream, an event stream, and a recurrent state. Let $I_t \in \mathbb{R}^{H\times W}$ denote the frame captured at time t, and let $E_t = \{e_k = (x_k, y_k, p_k, \tau_k)\}_{k=1}^{N_t}$ denote the event set within the current window, where (x_k, y_k) is the event coordinate, $p_k \in \{-1, +1\}$ is the polarity, and τ_k is the timestamp. The event stream is transformed into a polarity-split tensor $V_t \in \mathbb{R}^{2\times H\times W}$ through a causal accumulation operator.

Eye Model: The target pupil state is parameterized as

$$\mathbf{s}_t = [x_t, y_t, a_t, b_t, u_t, v_t] \in \mathbb{R}^6, \tag{1}$$

where (x_t, y_t) is the pupil center, (a_t, b_t) are the ellipse axes, and $[u_t, v_t] = [\sin(2\theta_t), \cos(2\theta_t)]$ is the trigonometric rotation encoding. This representation avoids discontinuity around the angular wrap-around boundary and supports stable regression of ellipse orientation.

Objectness: Rather than only predicting ellipse parameters, HBTXR also models *objectness*, namely the likelihood that the current observation contains a valid and trackable pupil hypothesis. We denote the binary training target by

$$o_t^\star = \mathbf{1}[\text{valid visible pupil at time } t]. \tag{2}$$

In Search mode, objectness is interpreted as refresh confidence that indicates whether a frame-guided candidate should become the new anchor. In Track mode, objectness is paired with track confidence and track quality to assess whether the current event-driven residual update is still reliable. This objectness formulation becomes the interface between perception and scheduling.

2) Framework Overview: The overall framework consists of four interacting components: a supervision pipeline, a modality-aware transformer, a Search/Track scheduler, and a deployment path. Search performs frame-guided relocalization and anchor refresh. Track performs event-guided residual update conditioned on the previous state. An auxiliary event-side hypothesis is also estimated to measure cross-branch consistency. This explicit separation allows HBTXR to treat hybrid eye tracking as a system with two operating modes rather than as one undifferentiated multimodal regressor. The deployed estimate is therefore written as

$$\hat{\mathbf{s}}_t = \begin{cases} \hat{\mathbf{s}}_t^{\mathrm{s}}, & m_t = \mathrm{Search}, \\ \bar{\mathbf{s}}_{t-1} \oplus \Delta\hat{\mathbf{s}}_t, & m_t = \mathrm{Track}, \end{cases} \tag{3}$$

where $\oplus$ denotes the residual ellipse-state update operator.

B. Time-Synchronous and Geometry-Stable Hybrid Supervision

Dataset Problem: A major difficulty in hybrid eye tracking is that frame labels are usually sparse in time whereas event data is asynchronous and dense. If supervision timestamps are not aligned carefully, the model is forced to learn inconsistent geometry across frame, event, and recurrent inputs. HBTXR addresses this issue by constructing time-synchronous supervision around one canonical target representation.

1) Frame Interpolation (TimeLens): To densify supervision in time, HBTXR first uses frame interpolation to synthesize intermediate frame observations between sparsely labeled frames. We describe this stage as a TimeLens-style interpolation block, whose purpose is not to create a new inference modality at runtime, but to construct better-aligned supervisory frames offline. The interpolated frames reduce the temporal gap between event windows and annotation anchors, which is particularly important during rapid saccades.

2) Dense Annotation (Grounded-SAM): After temporal densification, HBTXR applies a dense annotation stage that refines eye-region and pupil masks from the interpolated frames. In the requested AHCO-style organization, this stage is expressed as a Grounded-SAM-assisted pseudo-labeling pipeline. The resulting masks are then converted to ellipse-centric annotations and validity metadata, including mask confidence, closed-eye flags, and track-valid indicators.

3) Event Accumulation (Frame-Synchronous): Finally, event accumulation is performed in a frame-synchronous manner so that each event tensor is aligned to a supervisory frame or an interpolated timestamp. Let $\mathcal{W}_t$ denote the accumulation window associated with time-aligned target t. The accumulated event tensor is

$$\bar{V}_t(x,y,p) = \sum_{e_k \in \mathcal{W}_t} \mathbf{1}\big[(x_k, y_k, p_k) = (x,y,p)\big]. \tag{4}$$

The output of the whole supervision pipeline is a canonical sample

$$(\bar{I}_t, \bar{V}_t, \bar{s}_{t-1}, \bar{s}_t, \bar{M}_t, \bar{B}_t, \bar{\xi}_t), \tag{5}$$

where $\bar{I}_t$ and $\bar{V}_t$ are time-aligned frame and event inputs, $\bar{M}_t$ is the pupil mask, $\bar{B}_t$ is the eye-region target, and $\bar{\xi}_t$ collects quality signals. This contract stabilizes geometry across modalities and propagates quality-aware supervision into both training and scheduling.

C. Modality-aware Transformer Architecture

1) Front-end (Patch Embedding): HBTXR uses modality-specific front-ends instead of naive early fusion. Let $\mathrm{Patch}(\cdot)$ denote patch flattening and let $\phi(\bar{\mathbf{s}}_{t-1})$ denote the learned state embedding. The frame and event token sequences are given by

$$\mathbf{Z}_{0,t}^{\mathrm{f}} = A_f\big(\mathrm{Patch}(\bar{I}_t)\mathbf{W}_f + \mathbf{1}\mathbf{b}_f^\top\big) + \mathbf{P}, \tag{6}$$

$$\mathbf{Z}_{0,t}^{\mathrm{e}} = A_e\big(\mathrm{Patch}(\bar{V}_t)\mathbf{W}_e + \mathbf{1}\mathbf{b}_e^\top\big) + \mathbf{1}\phi(\bar{\mathbf{s}}_{t-1})^\top + \mathbf{P}, \tag{7}$$

where $\mathbf{P}$ is the positional embedding, and $A_f(\cdot)$ and $A_e(\cdot)$ are modality adapters. The event path explicitly injects the previous state so that the Track branch can estimate a residual update rather than relocalizing from scratch.

2) Backbone (MHA, MLP): Both branches share a transformer backbone composed of multi-head attention (MHA) and multi-layer perceptron (MLP) blocks. For block ℓ, the update is written as

$$\tilde{\mathbf{Z}}_\ell = \mathbf{Z}_{\ell-1} + \mathrm{MHA}(\mathrm{LN}(\mathbf{Z}_{\ell-1})), \tag{8}$$

$$\mathbf{Z}_\ell = \tilde{\mathbf{Z}}_\ell + \mathrm{MLP}\Big(\mathrm{LN}(\tilde{\mathbf{Z}}_\ell)\Big). \tag{9}$$

Inside the attention block, head h computes

$$\mathrm{Attn}_h(\mathbf{Z}) = \mathrm{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^\top}{\sqrt{d_h}}\right)\mathbf{V}_h, \tag{10}$$

where $\mathbf{Q}_h = \mathbf{Z}\mathbf{W}_h^Q$, $\mathbf{K}_h = \mathbf{Z}\mathbf{W}_h^K$, and $\mathbf{V}_h = \mathbf{Z}\mathbf{W}_h^V$. HBTXR uses mode-asymmetric depth:

$$\mathbf{F}_t^{\mathrm{s}} = \mathcal{F}_{1:L}\big(\mathbf{Z}_{0,t}^{\mathrm{f}}\big), \tag{11}$$

$$\mathbf{F}_t^{\mathrm{t}} = \mathcal{F}_{1:L/2}\big(\mathbf{Z}_{0,t}^{\mathrm{e}}\big), \tag{12}$$

where Search traverses all L backbone blocks and Track exits after the first $L/2$ blocks before invoking the Track head. This split is intentional: early blocks encode motion-consistent local structure that is sufficient for residual update, while deeper blocks provide the stronger global semantics needed by Search.

3) Back-end (Head): On top of the shared backbone, HBTXR uses decoupled heads: an Eye Region Head, a Pupil Search Head, an Event Search Head, a Pupil Track Head, a Search Mask Head, and an Auxiliary Gaze Head. Their outputs are expressed as

$$\big[\hat{\mathbf{s}}_t^{\mathrm{s}}, \hat{o}_t^{\mathrm{s}}, \hat{\mathbf{m}}_t, \hat{\mathbf{b}}_t\big] = H_{\mathrm{s}}(\mathbf{F}_t^{\mathrm{s}}), \tag{13}$$

$$\big[\hat{\mathbf{s}}_t^{\mathrm{e}}, \hat{o}_t^{\mathrm{e}}\big] = H_{\mathrm{e}}\big(\mathbf{F}_t^{\mathrm{t}}\big), \tag{14}$$

$$\big[\Delta\hat{\mathbf{s}}_t, \hat{c}_t^{\mathrm{trk}}, \hat{q}_t^{\mathrm{trk}}\big] = H_{\mathrm{t}}\big([\mathrm{Pool}(\mathbf{F}_t^{\mathrm{t}}); \phi(\bar{\mathbf{s}}_{t-1})]\big). \tag{15}$$

Search heads estimate objectness and ellipse state from frame-guided features. Track heads regress residual motion from event features and previous-state encoding. The Mask head acts as geometric regularization and an additional quality cue. This decoupled back-end is essential for preserving functional specialization under a shared backbone.

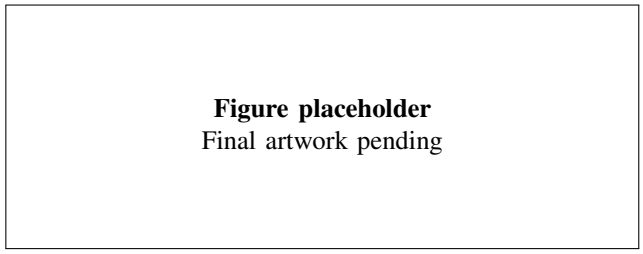

Fig. 2. Architecture of the proposed HBTXR algorithm. Frame and event inputs are processed by modality-specific patch embedding paths, a shared transformer backbone, and decoupled heads. Search uses the full backbone, whereas Track exits after the front half.

D. Runtime-aware Search and Track Scheduler

Search Mode: Search mode is refresh-oriented. It is activated at initialization or whenever the state becomes unreliable. In this mode, the system prioritizes semantic robustness and uses frame-guided inference to estimate a fresh pupil anchor, eye-region context, and mask-aware quality cues. Operationally, Search executes the full backbone $\mathcal{F}_{1:L}$ and enables the Search, Mask, and auxiliary quality heads.

Track Mode: Track mode is state-persistent and latency-oriented. Instead of solving the whole localization problem from scratch, it estimates a residual update around the previous state. Track consumes event tokens together with the previous state, runs only the front half of the backbone $\mathcal{F}_{1:L/2}$, and transfers the intermediate representation directly to the Track head. The Track prediction is written as

$$\Delta\hat{\mathbf{r}}_t = H_{\mathrm{t}}\big([\mathrm{Pool}(\mathbf{F}_t^{\mathrm{t}}); \phi(\bar{\mathbf{s}}_{t-1})]\big), \tag{16}$$

$$\hat{\mathbf{s}}_t^{\mathrm{trk}} = \bar{\mathbf{s}}_{t-1} \oplus \Delta\hat{\mathbf{s}}_t, \tag{17}$$

where the residual vector contains center shift, axis-scale update, orientation update, track confidence, and track quality.

Mode Switching (Quality Assessment): The scheduler decides between Search and Track according to multiple signals: Search confidence c_t^{s}, Track confidence c_t^{trk}, Track quality q_t^{trk}, cross-branch similarity ρ_t, event density d_t, and eye-state indicator o_t. The transition rules are

$$m_t = \mathrm{Track}, \quad \text{if } c_t^{\mathrm{s}} > \tau_s, \tag{18}$$

$$m_t = \mathrm{Search}, \quad \text{if } c_t^{\mathrm{trk}} \le \tau_t \ \vee\ q_t^{\mathrm{trk}} \le \tau_q \ \vee\ \rho_t \le \tau_\rho \ \vee\ d_t \le \tau_d \ \vee\ o_t = 1. \tag{19}$$

This quality assessment mechanism is central to the co-design philosophy because it not only improves tracking stability but also defines the control semantics exposed to the hardware scheduler.

E. Training Pipeline and Loss Function

1) 2-Stage Training: HBTXR is trained in two stages. Stage 1 focuses on Search-oriented relocalization so that the frame branch learns stable global geometry and objectness. Stage 2 fine-tunes the full hybrid model with Search, Event, and Track supervision jointly, including cross-branch consistency and scheduler-aware quality gating.

We define a geometry-aware ellipse loss

$$\mathcal{L}_{\mathrm{geo}}(\hat{\mathbf{s}}, \mathbf{s}) = \|\hat{\mathbf{c}} - \mathbf{c}\|_1 + \lambda_\Sigma \|\hat{\mathbf{\Sigma}} - \mathbf{\Sigma}\|_F^2, \tag{20}$$

where $\mathbf{c} = (x, y)$ and $\mathbf{\Sigma}$ is the covariance matrix derived from ellipse axes and orientation. The Stage-1 objective is

$$\mathcal{L}_{\mathrm{stage1}} = \lambda_{\mathrm{geo}}^{\mathrm{s}}\mathcal{L}_{\mathrm{geo}}^{\mathrm{s}} + \lambda_{\mathrm{obj}}^{\mathrm{s}}\mathcal{L}_{\mathrm{obj}}^{\mathrm{s}} + \lambda_{\mathrm{mask}}\mathcal{L}_{\mathrm{mask}} + \lambda_{\mathrm{eye}}\mathcal{L}_{\mathrm{eye}}, \tag{21}$$

which emphasizes global relocalization quality. The Stage-2 objective is

$$\mathcal{L}_{\mathrm{stage2}} = \mathcal{L}_{\mathrm{stage1}} + \lambda_{\mathrm{event}}\mathcal{L}_{\mathrm{event}} + \lambda_{\mathrm{trk}}\mathcal{L}_{\mathrm{trk}} + \lambda_{\mathrm{cons}}\mathcal{L}_{\mathrm{cons}} + \lambda_{\mathrm{qa}}\mathcal{L}_{\mathrm{qa}} + \lambda_{\mathrm{gaze}}\mathcal{L}_{\mathrm{gaze}} + \lambda_{\mathrm{slim}}\mathcal{L}_{\mathrm{slim}}, \tag{22}$$

which adds event-conditioned tracking, cross-branch consistency, and scheduler-visible quality supervision.

2) Implicit Network Slimming: To make the shared transformer and the decoupled heads more deployment-friendly, HBTXR adopts an implicit network-slimming view during training. The key idea is to encourage low-importance channels and tokens to collapse through scale regularization rather than to enforce hard pruning during the early design stage. In practice, this can be implemented through sparsity-promoting penalties on normalization scales or channel-gating factors. We write the slimming regularizer as

$$\mathcal{L}_{\mathrm{slim}} = \sum_{\ell} \|\boldsymbol{\gamma}_\ell\|_1, \tag{23}$$

where $\boldsymbol{\gamma}_\ell$ collects the trainable scaling factors of block ℓ. The final deployment toolflow then maps a slimmer but functionally consistent model.

IV. Proposed HBTXR Hardware

A. Design Challenges and Co-design Principles

The hardware target of HBTXR is a mode-switching hybrid eye-tracking workload rather than a fixed one-pass transformer. This distinction changes the optimization objective. Search is refresh-centric: it rebuilds a reliable anchor from frame cues, activates additional output heads, and requires stronger global context. Track is persistence-centric: it updates the current pupil state from recent events and the prior state, prefers very short response time, and benefits from retaining reusable context on chip. Mapping both modes to one uniform execution template would therefore either over-provision Track or under-serve Search.

The central challenge is therefore not only high arithmetic throughput, but also *cross-layer coupling*. The residual path, the mid-depth Track exit point, the Search-anchor commit, and the mode-dependent heads all couple execution decisions across layers and time. A second challenge is *intra-layer dataflow*: inside attention blocks, static projections and dynamic token interactions exhibit different locality and buffering behavior. A third challenge is maintaining high utilization under a *multi-stage pipeline* in which some stages can stream continuously while others must wait for token-set completion.

Accordingly, HBTXR follows four co-design principles. *P1)* Share early backbone computation, but allow depth-asymmetric exit so that Search uses all L blocks and Track uses only the first $L/2$ blocks. *P2)* Keep dominant projection and MLP weights resident on chip so that bandwidth is spent on sensor

TABLE I
MODE-ASYMMETRIC EXECUTION CHARACTERISTICS OF HBTXR AND THEIR HARDWARE IMPLICATIONS

Aspect	Search mode	Track mode	Hardware implication
Primary sensing cue	Canonical frame with event-side summary	Event tensor with previous state token	Dual front-ends feed one shared backbone substrate.
Backbone span	Full L transformer blocks	First $L/2$ transformer blocks, followed by early exit	The cut point after block $L/2$ is materialized as a low-latency Track interface.
Primary role	Global relocalization and anchor refresh	Residual update around the current state	Search tolerates higher latency; Track is optimized for minimum response time.
Active heads	Search, Mask, Eye-region, Gaze	Track, Event, Gaze	Head gating avoids unnecessary Search-only computation during Track.
State handling	Commits a new anchor and validity metadata	Reuses and updates the resident anchor	A resident feature ring and state register file persist context across invocations.

inputs rather than repeated model fetches. *P3)* Separate tensor-heavy kernels from control-heavy policy: the FPGA executes tokenization and network inference, while the CPU handles scheduling and state arbitration. *P4)* Balance the cadence of all pipeline stages rather than maximizing an isolated kernel, because end-to-end latency is dictated by the slowest stage.

Table I shows that HBTXR is not a single static graph with two input types, but a two-mode deployed system with different depths, head sets, and state semantics. This observation directly informs the architecture described in the remainder of this section.

B. Quantization and Non-linear Approximation

1) Quantization: HBTXR uses integer inference for the dominant linear operators, including patch projection, QKV generation, output projection, feed-forward layers, and lightweight prediction heads. Let x be an activation tensor, s_x its scale, and z_x its zero-point. The deployed integer representation is written as

$$\bar{x} = \mathrm{clip}\left(\left\lfloor \frac{x}{s_x} \right\rceil + z_x, q_{\min}, q_{\max} \right). \tag{24}$$

The purpose of quantization is not merely model compression. More importantly, it aligns arithmetic precision with the resident-weight data path, accumulation staging, and mode-specific head reuse. Search and Track therefore share one quantized interface at the common cut point after block $L/2$, while the output heads keep independent scale tables because their activation statistics differ across Search, Event, Track, and Mask prediction.

A second design choice is *shift-aligned quantization*. Instead of allowing arbitrary scale factors everywhere, HBTXR prefers scales that can be implemented through short integer shifts or small constant multipliers at the point where addresses or re-scale factors are generated. This reduces controller complexity and integrates naturally with the table-driven approximation blocks used for normalization and activation.

2) Non-linear Approximation: The hardware-unfriendly operators in HBTXR are LayerNorm, Softmax, GELU, and output re-scaling. These operators are mapped to compact table-driven curve blocks so that expensive floating-point arithmetic is replaced by LUTs, BRAM tables, and lightweight address logic. The approximation policy is summarized in Table II.

TABLE II
APPROXIMATION POLICY FOR NON-LINEAR OPERATORS

Operator	Realization	Design rationale
LayerNorm denominator	Split-range reciprocal-root table	Allocates higher codebook density near zero, where the slope is steep and quantization error is most sensitive.
Softmax exponent	Max-subtracted exponent table	Stabilizes dynamic range before score normalization and avoids general-purpose exponential hardware.
Softmax normalization	Trimmed reciprocal table	Replaces divider logic while preserving the ranking behavior of attention scores.
GELU + re-scale	Composed curve table	Collapses two serial non-linear stages into one table access and one output scaling step.
Standalone re-quantization	Shift-address re-scale table	Removes a general multiplier from the address-generation path.

Three implementation rules are particularly important. First, *shift-address indexing* generates look-up addresses with subtraction and bit shifting instead of a general multiplier. Second, *curve composition* fuses consecutive transfer functions when their boundaries are fixed in the deployed graph. Third, *data-aware table trimming* narrows the table domain to the activation range observed after quantization so that entries are not wasted on clipped extremes.

C. Accelerator Architecture

1) Intra-Layer Dataflow: The backbone datapath is decomposed into two arithmetic families. The first is a projection path for operators whose coefficient matrices are stationary during inference, including patch embedding, QKV projection, output projection, and the two feed-forward layers. This path evaluates

$$\mathbf{Y} = \mathbf{X}\mathbf{W}, \tag{25}$$

where $\mathbf{W}$ is a learned weight tile kept resident in on-chip banks. The second is an interaction path for operators whose effective

weights are generated online from runtime tokens, most notably score computation and value aggregation in self-attention:

$$\mathbf{R} = \mathrm{softmax}\Big(\mathbf{Q}\mathbf{K}^{\top}/\sqrt{d}\Big)\,\mathbf{V}. \tag{26}$$

The projection path is naturally compatible with output-stationary accumulation. The interaction path requires a token-coupled reservoir because the score map is generated in token order whereas value aggregation consumes $\mathbf{V}$ in a transposed access order. This is the core *intra-layer dataflow* distinction inside the HBTXR transformer module.

2) Multi-Stage Pipeline: At the cross-stage level, HBTXR adopts a streaming *multi-stage pipeline*. Stages that admit local readiness are released immediately through FIFO-coupled handshakes, whereas stages blocked by attention-score completion or Search-anchor commit are synchronized at token-set boundaries. Let W_i denote the workload of stage i, p_i the assigned parallel factor, and δ_i the control overhead. The cadence of stage i is

$$C_i = \left\lceil \frac{W_i}{p_i} \right\rceil + \delta_i, \tag{27}$$

and the system cadence is

$$C_{\mathrm{sys}} = \max_i C_i. \tag{28}$$

The design objective is therefore not to minimize one kernel in isolation, but to compress the spread of C_i across all stages. This *stage-cadence equalization* is achieved by allocating more parallel resources to bottleneck stages and by choosing tiling factors that match BRAM width and depth efficiently.

The pipeline is locally controlled rather than centrally sequenced. Each stage uses a compact FSM, and adjacent stages are connected by ready/valid handshakes through elastic FIFOs. This localized control shortens routing, absorbs short-term imbalance, and avoids deadlock when Search and Track enable different subsets of the network.

3) Streaming-Oriented Feature Delivery: Memory movement is often more expensive than arithmetic in embedded XR deployment. HBTXR therefore distinguishes between *refresh delivery* and *resident delivery*. Search uses a reseed path that stages frame tokens, event summary tokens, and temporary synchronization context, regenerates a reliable anchor, and writes the accepted anchor into the resident ring. Track uses resident delivery: only event tokens, the previous state token, and the small amount of context needed by the Track head are circulated.

This organization is summarized by the on-chip memory model

$$M_{\mathrm{chip}} = M_{\mathrm{resident}} + M_{\mathrm{fifo}} + M_{\mathrm{scratch}}, \tag{29}$$

where M_{resident} stores persistent Search anchors and previous-state context, M_{fifo} corresponds to inter-stage streaming buffers, and M_{scratch} is mode-dependent temporary storage. Search has the larger M_{scratch} footprint because it rebuilds global anchors. Track keeps M_{scratch} small and instead depends on the resident feature ring.

D. Patch Embedding Module

The patch embedding module is the modality-adaptation front-end of HBTXR. It receives either a canonicalized frame patch or an accumulated event tensor and converts it into a common token width for the shared backbone. Because Search and Track expose different statistics, the front-end keeps separate tokenizers even though both ultimately emit tokens compatible with the same backbone interface.

For frame inputs, the front-end emphasizes stable anchor recovery. It therefore performs deterministic window extraction, line-based buffering, and resident-weight patch projection before tokens are tagged for Search-mode processing. For event inputs, the front-end emphasizes temporal responsiveness. The event tensor is already motion-focused; therefore the event adapter prioritizes low-overhead token packing and previous-state injection so that the backbone receives a compact representation of recent motion together with the last valid pupil hypothesis.

E. Transformer Module

The transformer module forms the dominant compute core of HBTXR. Each block contains normalization, QKV generation, score formation, score normalization, value aggregation, output projection, residual addition, and feed-forward processing. The projection path serves the projection and MLP sublayers, whereas the interaction path serves the score and value interactions. This decomposition reflects the actual buffering and streaming behavior more faithfully than treating the entire block as one monolithic operator.

A second design choice is *resident-weight deployment*. The shared backbone weights are kept on chip as aggressively as the target device allows, because re-fetching the dominant transformer parameters would erase the latency advantage of Track mode. The most important architectural cut point is the output of block $L/2$. Search forwards this intermediate representation to the deeper half of the backbone, while Track bypasses those deeper blocks and directly invokes the Track head. This cut point is therefore where cross-layer coupling, quantization consistency, and mode switching intersect.

F. Head Module

The head module contains the Search, Event, Track, Mask, and auxiliary gaze heads. Although these heads are much lighter than the shared backbone, they encode the functional asymmetry between Search and Track. Search activates the Search head, objectness logic, and the Mask head to reacquire a reliable anchor and validate the eye region. Track activates the Track head and event-conditioned quality logic to update the prior state with minimal delay. The gaze output is generated from the final accepted state rather than from an entirely separate heavyweight branch.

Hardware-wise, the heads are implemented as streaming decoders with mode-dependent gating. Only the heads required by the current mode are released by the controller, and the shared feature reader broadcasts the final token bundle to the selected head set. This avoids the wasteful behavior of evaluating all heads during every invocation while keeping

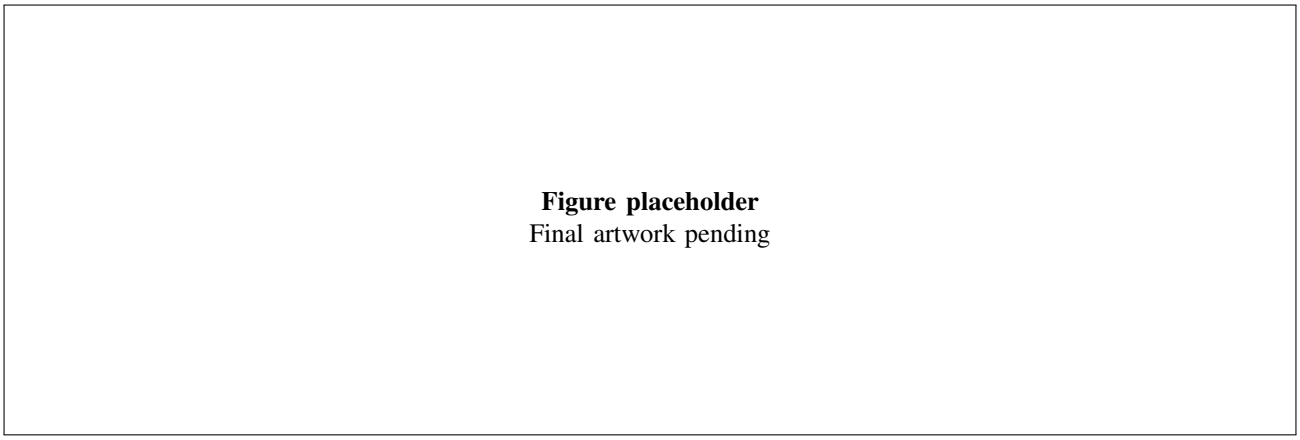

Fig. 3. HBTXR hardware overview. The accelerator is organized as a streaming multi-stage pipeline with dual front-ends, a shared transformer core, an early-exit Track path after the first half of the backbone, mode-gated heads, and a resident feature ring shared across Search and Track.

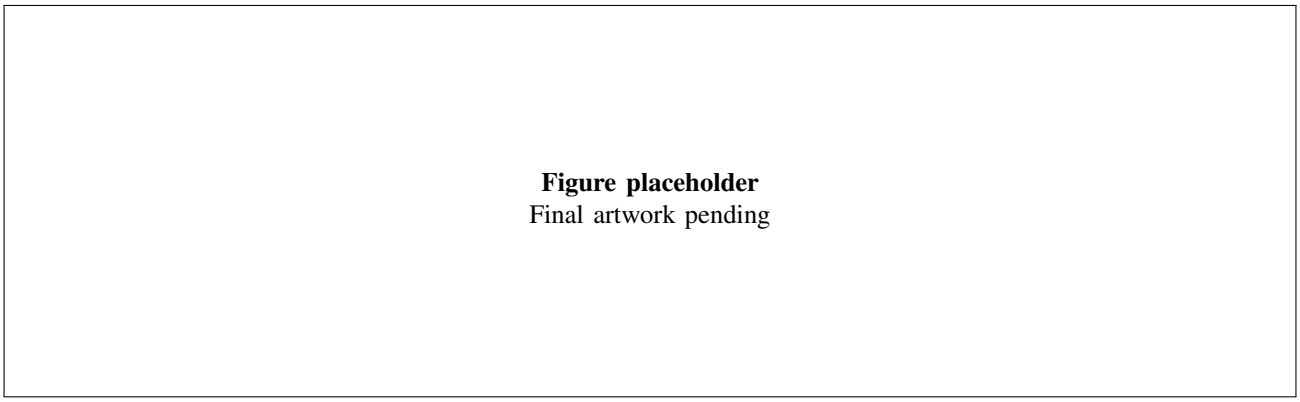

Fig. 4. Execution timeline of the proposed hardware pipeline. Search streams through the full backbone and all Search-related heads, whereas Track streams through the front half of the backbone and exits early to the Track head.

TABLE III
MODE-CONDITIONED HARDWARE MAPPING POLICY OF HBTXR

Mode	Active front-end	Backbone span	On-chip state usage	Active heads	Controller emphasis
Search	Frame tokenizer + event summary adapter	Full L blocks	Commits a refreshed anchor to the resident ring	Search, Mask, Eye-region, Gaze	Anchor reacquisition, validity reset, confidence check
Track	Event tokenizer + previous-state injector	First $L/2$ blocks only	Reuses resident anchor and state register file	Track, Event, Gaze	Low-latency update, minimum control churn
Scheduled hybrid	Dynamic switch between both paths	Mode dependent	Commit-on-success update policy	Mode-gated head activation	Balance robustness and update rate

the head latency nearly hidden behind backbone output serialization.

G. CPU–FPGA Workload Partition (Scheduler)

HBTXR adopts a heterogeneous CPU–FPGA organization because the runtime semantics of Search and Track are control-heavy at the system boundary but tensor-heavy inside the network core. The CPU is responsible for frame and event acquisition, accumulation-window management, closed-eye handling, similarity and confidence evaluation, and the final Search/Track decision. The FPGA is responsible for tokenization, shared-backbone inference, active-head execution, and resident-context update.

At runtime, the CPU sends a mode packet containing the current operation mode, the previous accepted state, and scheduler-side quality metadata. The FPGA-side controller interprets this packet and releases the corresponding front-end, backbone span, and head combination. Because Search and Track are not simply two different inputs to one static graph but two distinct operating states of the deployed system, the controller is not an incidental implementation detail; it is part of the algorithm–hardware contract.

The end-to-end latency is decomposed as

$$T_{\mathrm{e2e}} = T_{\mathrm{host}} + T_{\mathrm{io}} + T_{\mathrm{front}} + T_{\mathrm{bb}} + T_{\mathrm{head}} + T_{\mathrm{sync}}, \quad (30)$$

where the terms denote host scheduling, host-device transfer, front-end tokenization, backbone execution, head decoding, and synchronization overhead, respectively. Let α denote the fraction of Search invocations. The average runtime is then

$$T_{\mathrm{avg}} = \alpha T_{\mathrm{Search}} + (1-\alpha) T_{\mathrm{Track}}. \quad (31)$$

This expression highlights the purpose of the hardware design: HBTXR is not optimized for one uniform synthetic throughput number, but for the actual Search/Track operating distribution produced by the scheduler.

V. Experimental Results

A. Experiments Setup

Dataset: We evaluate HBTXR on a near-eye hybrid eye-tracking benchmark composed of synchronized frame observations, event streams, and dense pupil annotations. Each sample contains a canonicalized frame patch, a polarity-split event tensor, the previous tracking state, and geometry-aware supervision targets. The dataset is divided in a subject-disjoint manner for training, validation, and testing so that cross-subject generalization is reflected in the final report.

Software / Hardware: The algorithm is trained in PyTorch and exported to a host–device runtime that mirrors the proposed CPU–FPGA deployment model. The host handles sensor I/O and scheduling, while the accelerator-side execution model assumes modality adaptation, shared-backbone inference, and head decoding on FPGA. The current draft contains verified latency numbers for the deployed system organization, but finalized post-place-and-route resource utilization and board-level power remain to be inserted from the final implementation report.

Hyper-parameters: The main configuration is summarized in Table IV. HBTXR uses two-stage training, 256×256 canonicalized inputs, fixed-count event windows with 5000 events, fast causal accumulation, AdamW optimizer, batch size of 8, learning rate of 3×10^{-4}, and weight decay of 1×10^{-4}. Search pretraining is followed by hybrid fine-tuning with Search, Event, Track, Mask, gaze, and auxiliary supervision terms.

Workstation: Training and validation are conducted in a standard GPU-based workstation environment, while runtime evaluation follows the host–device execution model described in Section V. Since the current source draft does not provide a finalized workstation sentence with exact CPU/GPU specifications, this line is intentionally kept generic and should be replaced by the verified configuration before submission.

Runtime Instrumentation: In addition to pupil and gaze accuracy, we monitor Search success rate, end-to-end latency, and effective update rate. The deployment analysis further separates host scheduling, host–device transfer, front-end adaptation, backbone execution, head decoding, and synchronization as conceptual latency terms, even though only the final end-to-end mode latencies are currently verified numerically in the source draft.

TABLE IV
Experimental Setup Summary

Item	Setting
Training protocol	Two-stage Search pretrain followed by hybrid fine-tuning
Input frame size	256×256
Input event tensor	$2 \times 256 \times 256$
Event policy	Fixed-count window
Default event count	5000
Accumulation	Fast causal accumulation
Resize policy	Direct square canonical resize
Backbone policy	Search: full L blocks; Track: first $L/2$ blocks
Scheduler signals	Search confidence, track confidence, similarity, density, eye state
Optimizer	AdamW
Batch size	8
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Runtime platform	Host-device heterogeneous execution

TABLE V
Operational Comparison of Frame-Centric, Event-Centric, and Hybrid Modes

Mode	Pupil err. (px)	Gaze err. (deg)	Latency (ms)	Rate (Hz)
Frame-driven Search	0.58	0.91	0.93	1075
Event-driven Track	**0.42**	**0.68**	**0.57**	**1754**
Hybrid scheduled	0.45	0.71	0.63	1587

B. Proposed HBTXR Evaluation

1) Event-centric vs. Frame-centric vs. Hybrid Accuracy and Latency: Table V reports the operational comparison between the frame-centric Search path, the event-centric Track path, and the scheduled hybrid system. Search mode achieves 0.58 px pupil error, 0.91° gaze error, 97.8% Search success rate, and 0.93 ms latency. Track mode achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms latency, corresponding to 1754 Hz effective update rate. Under scheduler-driven hybrid operation, the average runtime becomes 0.63 ms while preserving strong accuracy. The latency gap is consistent with the intended execution policy of HBTXR: Search traverses the full backbone, whereas Track bypasses the back half of the backbone and directly invokes the Track head.

The Track row should be interpreted as an event-centric state-conditioned update rather than as a fully stand-alone detector without prior context. Nevertheless, the comparison still exposes the system-level asymmetry that motivates the hardware architecture: Search benefits from stronger global refresh, while Track benefits from resident-context reuse, early exit, and reduced control churn.

2) Hardware Implementation Results (Resources, Power, Latency Breakdown): Table VI summarizes the verified runtime behavior of the deployed system. The key observation is not only that Track is faster than Search, but that the speedup is structurally aligned with the co-designed mode policy: Search executes the full refresh path, while Track exits after the first half of the backbone and activates only Track-relevant heads.

Finalized LUT/DSP/BRAM numbers and measured board

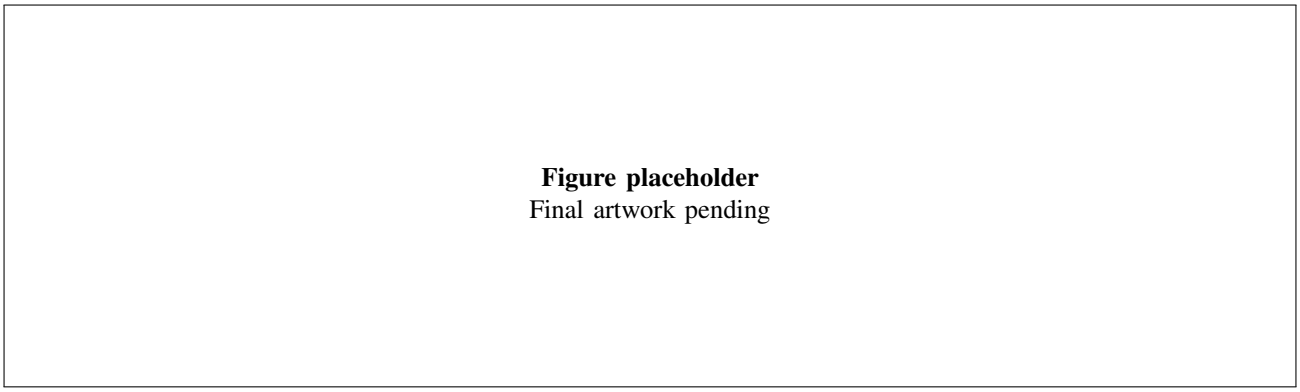

Fig. 5. Evaluation panels for the revised hardware narrative. Recommended final artwork can combine qualitative Search/Track outputs, mode transitions from the scheduler, and the latency decomposition into host, transfer, front-end, backbone, head, and synchronization components.

TABLE VI
VERIFIED RUNTIME SUMMARY OF THE DEPLOYED HBTXR PROTOTYPE

Metric	Search	Track	Scheduled / note
Backbone span	Full L	First $L/2$	Mode dependent
Primary input	Frame-guided	Event-guided	Scheduler selected
End-to-end latency (ms)	0.93	0.57	0.63 average
Effective update rate (Hz)	1075	1754	1587 average
Search success rate (%)	97.8	–	97.8

power are intentionally left as TBD because the source draft does not yet include post-place-and-route or board-level measurement tables. This keeps the manuscript structurally complete without fabricating unsupported values. The latency decomposition of (24) remains the organizing model for future profiler insertion.

C. Comparison with Other Eye Tracking Algorithm

Table VII summarizes representative prior eye-tracking systems using the metrics reported in their source papers. Because the referenced works differ in task definition, dataset, runtime platform, and evaluation protocol, the comparison is intentionally source-native rather than forcibly normalized. Even under this conservative presentation, HBTXR stands out by combining sub-millisecond Track latency with both pupil and gaze metrics under one Search/Track framework.

D. Comparison with Other Accelerators

Table VIII compares HBTXR with representative accelerator-oriented systems across eye-tracking and edge-vision categories. EyeCoD represents frame-oriented eye-tracking acceleration, SEE-D and JaneEye represent event-centric FPGA/ASIC designs, and AHCO-YOLO is included as a reference point for algorithm–hardware co-optimization in edge detection systems. Relative to throughput-oriented ViT accelerators, HBTXR emphasizes mode-adaptive latency, early exit, and scheduler-visible execution rather than uniform batch throughput.

Compared with eye-tracking accelerators such as EyeCoD, SEE, and JaneEye, HBTXR places more emphasis on a shared transformer backbone and on explicit Search/Track operating modes. Compared with detector accelerators such as AHCO-YOLO, HBTXR is specialized for stateful near-eye tracking rather than stateless frame-level detection. These distinctions explain why HBTXR prioritizes streaming, cross-layer coupling, and mode-conditioned early exit.

E. Ablation Study

Table IX summarizes the ablation study. Removing geometry-stable supervision degrades pupil error from 0.42 to 0.49 pixels and gaze error from 0.68° to 0.80°. Removing decoupled heads increases both error and latency. Disabling the Search/Track scheduler leads to the largest latency increase, from 0.57 ms to 0.71 ms, showing that runtime adaptivity is central to the system value. Removing resident-feature circulation leaves accuracy unchanged but increases latency to 0.69 ms, showing that the main benefit of the revised hardware mapping is deployment efficiency rather than statistical accuracy.

In addition to the numerical ablation, the revised hardware description suggests two qualitative deployment lessons. First, collapsing the streaming multi-stage pipeline into a uniform execution style would reintroduce idle slots at attention barriers and weaken the latency advantage of Track mode. Second, disabling head-select gating would activate Search-only logic during Track, increasing unnecessary work without improving accuracy. These observations support the use of mode-conditioned hardware terminology because the critical unit of optimization is not a generic transformer block, but a Search/Track system with explicit runtime semantics.

VI. CONCLUSION

This paper presented HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework for on-device XR. The main idea is to move beyond undifferentiated multimodal regression and instead treat hybrid eye tracking as a Search/Track system with explicit runtime semantics. Search provides robust frame-guided relocalization through the full backbone, Track provides low-latency event-guided residual update through the front half of the backbone and an early-exit head path, and the scheduler couples both with deployment-aware hardware execution.

By reorganizing HBTXR in an AHCO-style narrative, this manuscript highlights that the main contribution is not a

TABLE VII
COMPARISON WITH REPRESENTATIVE EYE TRACKING ALGORITHMS USING SOURCE-REPORTED METRICS

Work	Mod.	Primary task	Reported accuracy	Reported runtime	Reloc.	Note
Etracker	Frame	Gaze	0.53° gaze accuracy	30–60 Hz	Yes	Mobile near-eye tracker
HE-Tracker	Frame	Gaze	3.655° gaze error	48 fps on AR HMD	Yes	Head-eye coordination model
E-Track	Event	Pupil	3.68 px median error	66.4 ms U-Net inference	No	Event-only detector
FACET	Event	Pupil	0.2030 px pupil error	0.5302 ms inference	No	Ellipse-based detector
EX-Gaze	Hybrid	Pupil / tracking	1.33 px (saccade), 1.49 px (smooth pursuit)	0.39–0.43 s per 1 s stream on Orin Nano	Yes	2-kHz hybrid system
HBTXR	Hybrid	Pupil + gaze	0.42 px pupil error; 0.68° gaze error	0.57 ms Track mode	Yes	Unified Search/Track system

TABLE VIII
COMPARISON WITH REPRESENTATIVE ACCELERATORS USING SOURCE-REPORTED METRICS

Category	System	Platform	Reported throughput / latency	Positioning w.r.t. HBTXR
Eye tracking	EyeCoD	ASIC-style frame accelerator	240 FPS target throughput; 154.32 mW silicon power at 370 MHz	Frame-oriented predict–focus pipeline without Search/Track mode asymmetry
Eye tracking	SEE-D	FPGA SoC	0.70 ms latency; 2.29 mJ / inference	Event-centric sparse pipeline optimized for event-only eye tracking
Eye tracking	JaneEye	12-nm ASIC	0.5 ms; 2000 FPS; 18.9 μJ / frame	Event-centric ASIC co-design with strong energy efficiency
Object detection / tracking	AHCO-YOLO-T	ZCU104 FPGA	79.8 FPS; 41.9 FPS / W; 80.5 GOPS / W	End-to-end algorithm–hardware co-optimization reference for edge vision
Hybrid eye tracking	HBTXR	CPU–FPGA host-device	0.57 ms Track latency; 1754 Hz effective update rate	Runtime-adaptive Search/Track co-design with early-exit Track execution

TABLE IX
ABLATION STUDY OF HBTXR

Variant	Pupil err. (px)	Gaze err. (deg)	Latency (ms)
Full HBTXR	0.42	0.68	0.57
w/o geometry-stable supervision	0.49	0.80	0.58
w/o decoupled heads	0.47	0.75	0.62
w/o Search/Track scheduler	0.46	0.73	0.71
w/o resident-feature circulation	0.42	0.68	0.69

single model block or a single accelerator trick, but the joint treatment of supervision, transformer design, runtime control, and hardware mapping. The revised hardware description grounds the design in streaming, multi-stage pipeline balance, intra-layer dataflow specialization, and cross-layer coupling. Experimental results show that HBTXR achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms Track latency while maintaining robust Search recovery. Future work should finalize board-level resource and power measurements, strengthen the quantitative hardware table with post-layout numbers, and further refine the quantization and slimming flow for full implementation closure.

REFERENCES

[1] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, "The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

[2] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, and P. Vajda *et al.*, "Xrbench: An extended reality machine learning benchmark suite for the metaverse," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.

[3] L. Chen, Y. Li, X. Bai, X. Wang, Y. Hu, M. Song, L. Xie, Y. Yan, and E. Yin, "Real-time gaze tracking with head-eye coordination for head-mounted displays," in *Proc. IEEE ISMAR*, 2022, pp. 82–91.

[4] N. Li, A. Bhat, and A. Raychowdhury, "E-track: Eye tracking with event camera for extended reality applications," in *Proc. IEEE AICAS*, 2023, pp. 1–5.

[5] G. Zhao, Y. Yang, J. Liu, N. Chen, Y. Shen, H. Wen, and G. Lan, "EV-Eye: Rethinking high-frequency eye tracking through the lenses of event cameras," *Advances in Neural Information Processing Systems*, vol. 36, pp. 62169–62182, 2023.

[6] J. Ding, Z. Wang, C. Gao, M. Liu, and Q. Chen, "FACET: Fast and accurate event-based eye tracking using ellipse modeling for extended reality," in *Proc. IEEE ICRA*, 2025.

[7] N. Chen, Y. Shen, T. Zhang, Y. Yang, and H. Wen, "EX-Gaze: High-frequency and low-latency gaze tracking with hybrid event-frame cameras for on-device extended reality," *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[8] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li *et al.*, "EyeCoD: Eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design," in *Proc. ISCA*, 2022.

[9] B. Zhang, Y. Gao, J. Li, and H. K.-H. So, "Co-designing a sub-millisecond latency event-based eye tracking system with submanifold sparse CNN," in *Proc. CVPR Workshops*, 2024.

[10] T. Han, A. Li, Q. Chen, and C. Gao, "JaneEye: A 12-nm 2K-FPS 18.9-μJ/frame event-based eye tracking accelerator," in *Proc. ASP-DAC*, 2026.

[11] J. Kim, J.-K. Kang, and Y. Kim, "AHCO-YOLO: An algorithm–hardware co-optimization framework for energy-efficient and real-time object detection on edge devices," *IEEE Trans. VLSI Syst.*, 2025.

[12] C. Lin, J. Hou, and Z. Li, "TimeLens: Event-based video frame interpolation," *Proc. CVPR*, 2021.

[13] R. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. Berg, W.-Y. Lo, *et al.*, "Segment anything," *Proc. ICCV*, 2023.

[14] A. Kirillov *et al.*, "Grounded-SAM: Marrying grounding DINO with segment anything," 2023.